

U1T01: Real-Time Analytics with PostgreSQL CDC & ClickHouse

Team: Julio Cesar De Aquino Castellanos (relational modeling & data generator), Valeria Nicol Hernandez Leon (CDC pipeline / PeerDB), Jose Angel Pech Xool (ClickHouse analytical layer), Leonora del Carmen Horta Sanchez (benchmarking), Lorena Danae Perez Lopez (real-time dashboard), Gustavo Alexander Fuentes Marin (documentation).

1. Overview

We implemented Change Data Capture (CDC) from PostgreSQL (OLTP) to ClickHouse (OLAP) using PeerDB, fed by a continuous Python data generator, and benchmarked equivalent analytical queries on both engines. An optional Streamlit dashboard visualizes ClickHouse data in near real time.

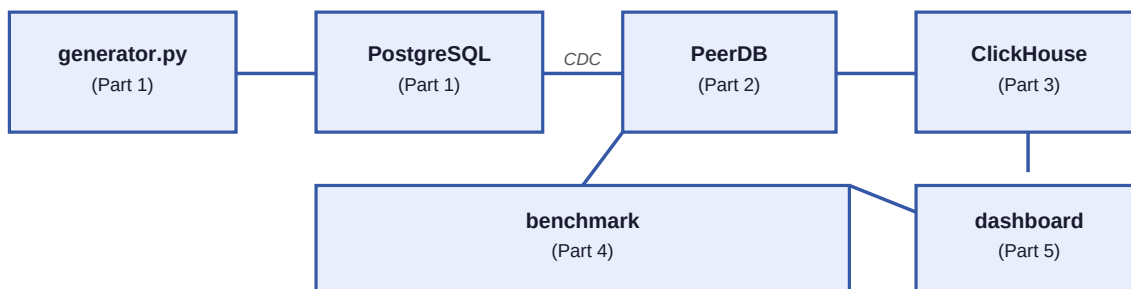


Figure: data flow between parts

The whole stack (Postgres, the generator, the full PeerDB deployment — Temporal, MinIO, catalog, flow workers, UI — ClickHouse, and the dashboard) runs with a single `docker compose up --build`, no manual setup step required. This was verified end to end for this report: `peerdb-init` logged `CDC setup completed!`, every service came up healthy, and row counts were observed flowing live from Postgres into ClickHouse while the generator ran, with the dashboard serving HTTP 200 on port 8501.

2. Part 1 — Relational Model & Data Generator

2.1 Domain and schema (3NF)

We modeled an e-commerce domain: it produces a natural mix of INSERT (new orders), UPDATE (status transitions), and DELETE (cancelled/removed items) operations, and the analytical questions (revenue by category, top products, AOV, payment success) are straightforward to define and interpret.

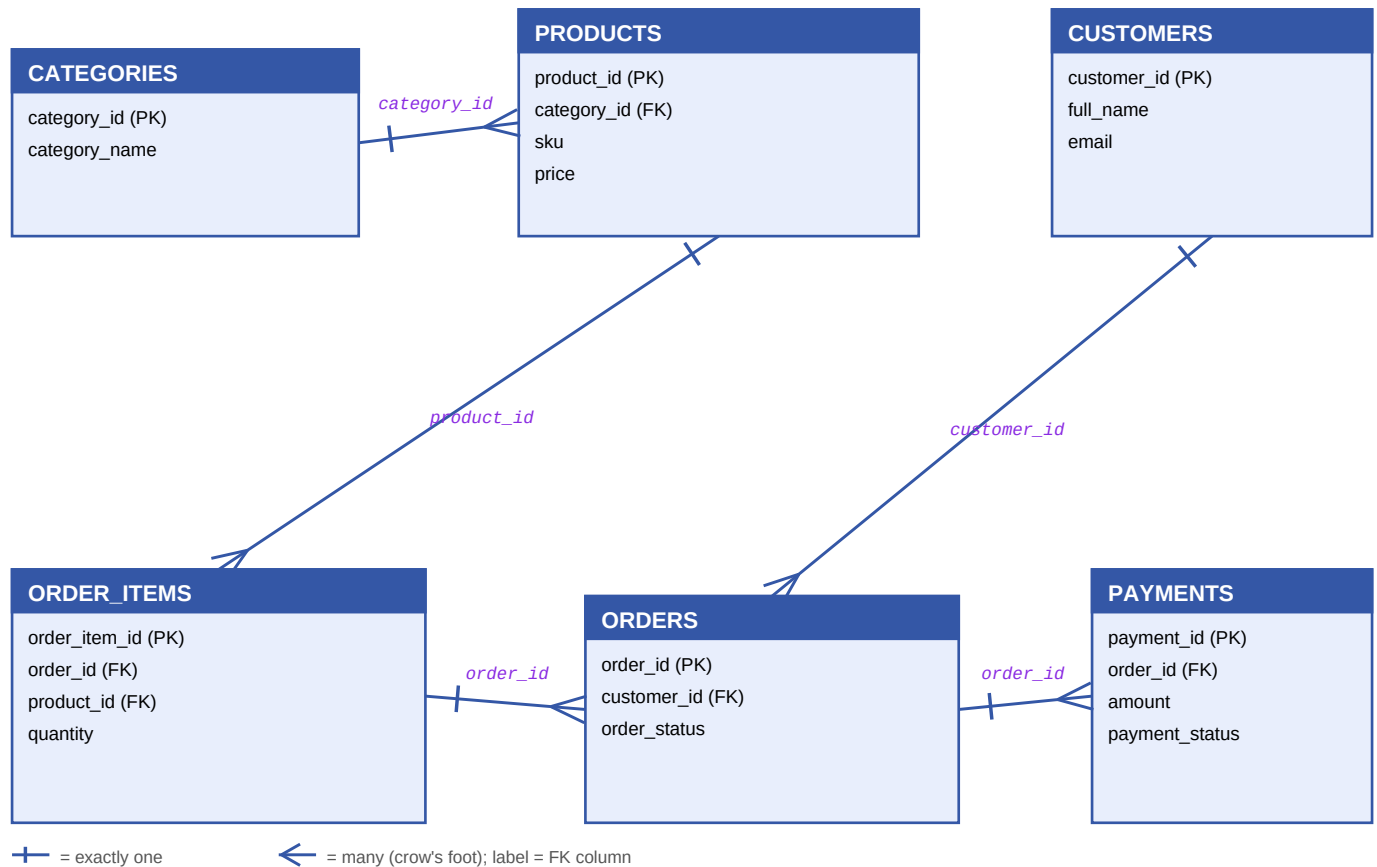


Figure: entity-relationship diagram (6 tables, 3NF)

Six tables, all in third normal form: every non-key attribute depends on the whole primary key and nothing but the primary key. `order_items.unit_price` stores a price **snapshot** at purchase time — this is not a normalization violation, since it is a fact that cannot be derived later (a product's price changes over time, but the order must remember what was actually charged). Every mutable table carries `updated_at`, which the generator bumps on every UPDATE.

PostgreSQL enforces the model with primary keys, foreign keys, and `CHECK` constraints on the enum-like columns (`order_status`, `payment_status`, `payment_method`), plus indexes on all foreign-key columns for join performance.

2.2 PostgreSQL configuration for CDC

Logical replication was enabled at container start (`wal_level=logical`, set via the Postgres command line — it cannot be changed with `ALTER SYSTEM` after the fact without a restart). A dedicated `replicator` role was created with the `REPLICATION` privilege and `SELECT` on all six tables, and a publication `peerdb_pub` was created covering all six tables — this is what PeerDB subscribes to.

2.3 Data generator

A Python script (`generator/generator.py`) seeds reference data (categories, products, customers) once, then loops indefinitely firing a weighted mix of operations: creating orders with 1–5 items and a payment; advancing order status through `pending` → `paid` → `shipped` → `delivered` (or `pending` → `cancelled`); advancing payment status through `pending` → `completed` or `pending` → `failed` → `refunded`; editing customer/product fields; and deleting a pending order item or a stale failed payment — so the DELETE replication path

is genuinely exercised, not just simulated as a status change.

Verification (captured from a real run):

- All 6 tables created; `wal_level=logical`; publication `peerdb_pub` covers all 6 tables with inserts/updates/deletes/truncates enabled; `replicator` role has the `Replication` attribute.
- Row counts measured twice, 15 seconds apart: `order_items` +264, `orders` +95, `payments` +91 — confirms continuous write traffic.
- A query isolating rows where `updated_at > order_date` confirms real UPDATES are firing (not just fresh inserts getting both timestamps at once).
- The generator handles restarts idempotently: on SIGTERM it prints final totals, and on the next start it detects existing reference data and skips reseeding.
- Full transcript with exact commands and output: [postgres/README.md](#).

Workload achieved: across the runs used for this report, the generator produced **202,597 total INSERT/UPDATE/DELETE mutations** against PostgreSQL (an initial one-hour run plus an extension at 100 events/sec), exceeding the assignment's target of a couple hundred thousand operations.

3. Part 2 — CDC Pipeline Setup (PeerDB)

PeerDB's stack (catalog database, Temporal for workflow orchestration, MinIO, flow API, flow workers, the PeerDB server, and its UI) is integrated directly into the root `docker-compose.yml`, all services prefixed `peerdb-` to avoid collisions with the project's own services.

A `peerdb-init` container automates mirror creation: it waits for the PeerDB server's SQL-compatible endpoint (port 9900) to come up, then registers PostgreSQL and ClickHouse as peers and creates a mirror named `postgres_to_clickhouse` that consumes the `peerdb_pub` publication from Part 1. No manual UI step is required — this was a deliberate requirement, since a grader running `docker compose up` should not have to click through a UI to get CDC flowing. Mirror configuration:

- `do_initial_copy = true` — existing rows (seeded before the mirror is created) are copied too.
- `sync_interval = 10` seconds — changes reach ClickHouse within seconds.
- `PEERDB_NULLABLE = true` — columns that accept `NULL` in Postgres (e.g. `payments.paid_at`, `customers.city/country`) are created as `Nullable(...)` in ClickHouse instead of PeerDB's default of writing epoch/empty placeholder values.

Mirror status can be checked via the PeerDB UI (<http://localhost:3000>, showing live metrics for `postgres_to_clickhouse`) or via PeerDB's Postgres-compatible SQL endpoint on port 9900 (`SELECT name, status FROM flows;`).

Key finding — delete propagation: PeerDB does not perform a hard delete on the ClickHouse side. Deletes are propagated as **soft deletes**: PeerDB adds three columns to every destination table (`_peerdb_version` — a monotonically increasing version, `_peerdb_is_deleted` — 0/1, `_peerdb_synced_at` — sync timestamp), and a Postgres DELETE arrives in ClickHouse as a new row version with `_peerdb_is_deleted = 1`. Because the tables use `REPLICA IDENTITY DEFAULT`, Postgres only sends the primary key on delete, so every non-key column in that soft-delete row arrives empty (0, '', epoch date, or `NULL` where nullable). This is why every analytical query against ClickHouse must filter `_peerdb_is_deleted = 0` — this finding directly shaped the ClickHouse schema and query design in Part 3.

4. Part 3 — Analytical Layer (ClickHouse)

4.1 Schema

All six tables are recreated in ClickHouse with the exact column names and types PeerDB itself would generate (verified by letting PeerDB create tables once and comparing `SHOW CREATE TABLE`), so PeerDB reuses the pre-created tables rather than conflicting with its own inferred schema:

PostgreSQL	ClickHouse
<code>SERIAL / INTEGER</code>	<code>Int32</code>
<code>NUMERIC(10,2)</code>	<code>Decimal(10,2)</code>
<code>VARCHAR(n)</code>	<code>String</code>
<code>TIMESTAMPTZ</code>	<code>DateTime64(6)</code>
nullable column	<code>Nullable(...)</code>

Every table uses engine `ReplacingMergeTree(_peerdb_version)`, `ORDER BY` the Postgres primary key. This directly addresses the assignment's note that ClickHouse "may not handle updates and deletes exactly like PostgreSQL": ClickHouse's storage is built from immutable parts, so PeerDB represents every Postgres UPDATE or DELETE as a **new row** with the same key and a higher `_peerdb_version`. `ReplacingMergeTree` keeps only the highest-versioned row per key when parts are merged in the background — and since merges happen on no fixed schedule, queries must add `FINAL` to force that deduplication at read time and see the current state.

Why `_peerdb_version` and not `updated_at` (the version column originally proposed in the project's shared contract): on DELETE, Postgres sends only the primary key, so the soft-delete row's `updated_at` arrives as the epoch (`1970-01-01`) and would always lose against the real previous version — the deleted row would "come back to life" under `FINAL`. `categories` also has no `updated_at` column at all. `_peerdb_version` is the only column PeerDB guarantees to be monotonically increasing on every change, insert or delete alike.

4.2 Verification: UPDATE and DELETE reach ClickHouse correctly

An automated script (`clickhouse/verify.sh`) changes rows in Postgres and shows the resulting state in ClickHouse. Captured evidence:

UPDATE — `UPDATE customers SET email = '...' WHERE customer_id = 1;` in Postgres. Without `FINAL`, both the old and new row versions are visible in ClickHouse until background merge; with `FINAL`, only the latest version (highest `_peerdb_version`) is returned — confirmed within ~2 seconds of the Postgres update.

DELETE — `DELETE FROM payments WHERE payment_id = 1;` in Postgres. `SELECT ... FINAL` in ClickHouse returns the soft-delete tombstone (`_peerdb_is_deleted = 1`, all non-key columns empty) within ~11 seconds; adding the standard `WHERE _peerdb_is_deleted = 0` filter makes the row disappear entirely, as expected.

Generator-driven updates — of the last 5 orders the generator moved to `delivered/cancelled`, all 5 showed the identical `order_status` and `updated_at` in both PostgreSQL and `ecommerce.orders FINAL` in ClickHouse.

Full command-by-command transcript: `clickhouse/verification/latest_run.txt`.

4.3 Standard query pattern (used by Parts 4 and 5)

```
SELECT order_status, count() AS orders
FROM ecommerce.orders FINAL
WHERE _peerdb_is_deleted = 0
GROUP BY order_status;
```

Every table read in every query — including each side of a JOIN — carries `FINAL` and the `_peerdb_is_deleted = 0` filter. Omitting `FINAL` double-counts updated rows (stale + current versions both present); omitting the filter counts

soft-deleted rows whose values are empty placeholders.

5. Part 4 — Benchmarking

5.1 Methodology

Four analytical queries were written with equivalent business meaning in both SQL dialects ([benchmark/queries/*.pg.sql](#) / [*.ch.sql](#)):

1. **Revenue by category**, grouped by month. 2. **Top 10 products** by revenue. 3. **Average order value (AOV) by customer country**. 4. **Payment success rate by payment method**.

All four join across [orders](#), [order_items](#), [products/categories](#), or [payments](#) and aggregate/group/filter across the full dataset, per the assignment's requirement. They were run while the generator (and the CDC pipeline) were actively producing new writes — not against a static snapshot.

Because the generator can update an order or payment's status at any time, comparing "the current full dataset" between two systems that receive changes on different schedules (Postgres immediately, ClickHouse ~10s later via CDC) is not reproducible. The benchmark therefore restricts each query to a **terminal cohort** — rows whose lifecycle has settled ([delivered](#) orders; [completed/refunded](#) payments) as of a shared UTC cutoff timestamp that both engines had already fully replicated. This keeps the live-write requirement (the generator never stops) while making the compared result sets deterministic and verified equal between engines before timing began.

Each query/engine pair was run once, unmeasured, to confirm row-for-row equivalence (numeric tolerance of one cent, to absorb ClickHouse's floating-point average rounding), then measured over **5 repetitions per engine per query**, alternating engine order, using [EXPLAIN ANALYZE](#), [BUFFERS](#) on PostgreSQL and [EXPLAIN](#) on ClickHouse to capture plans (full plan files in [benchmark/results/](#)).

Workload at benchmark time: 202,597 total mutations had been applied to PostgreSQL by the generator ([GEN_RATE_PER_SEC=100](#), ~125 mutations/sec sustained) before the timed run, satisfying the assignment's live-ingestion requirement.

5.2 Results

Wall-clock time per query (milliseconds, end-to-end including client/container overhead), across the 5 measured repetitions:

Query	Postgres min	Postgres median	Postgres max	ClickHouse min	ClickHouse median	ClickHouse max
Revenue by category	219.07	298.02	386.57	283.97	388.74	427.31
Top products	259.58	307.00	336.52	335.68	358.31	384.75
AOV by country	216.49	349.55	365.73	219.87	302.61	342.60
Payment success by method	226.17	318.00	362.42	273.61	370.98	401.93

PostgreSQL had the lower median for revenue by category (23.3% lower), top products (14.3% lower), and payment success (14.3% lower). ClickHouse had the lower median for AOV by country (13.4% lower). **These differences reflect this specific dataset size, query shape, and CDC-read overhead (FINAL) — they are not a general verdict on which engine is "faster."**

5.3 Why the plans differ

PostgreSQL uses its row-store B-tree index (`idx_orders_status`) via a bitmap index scan + bitmap heap scan to find `delivered` orders, applies the date/ `updated_at` cutoff as a filter, sequentially scans the larger `order_items` fact table, and combines everything with hash joins, hash aggregation, and a final sort (top-products uses a top-N heapsort instead). Excerpt from the captured `EXPLAIN (ANALYZE, BUFFERS)` for revenue-by-category:

```
Hash Join (cost=328.07..2234.69 rows=3142 width=232) (actual time=1.578..22.182 rows=3818 loops=1)
  Hash Cond: (oi.product_id = p.product_id)
  -> Hash Join (cost=311.97..2159.68 rows=3142 width=22) (actual time=1.351..20.182 rows=3818 loops=1)
    Hash Cond: (oi.order_id = o.order_id)
    -> Seq Scan on order_items oi (cost=0.00..1605.52 rows=92252 width=18) (actual time=0.003..8.565 rows=92332 loops=1)
    -> Hash -> Bitmap Heap Scan on orders o
      Recheck Cond: ((order_status)::text = 'delivered'::text)
```

ClickHouse reads every CDC table through a `ReadFromMergeTree` node with `FINAL: 1`, meaning it must resolve `ReplacingMergeTree` versions and drop CDC tombstones (`_peerdb_is_deleted = 1`) *before* any aggregation can run — extra read-time work that PostgreSQL's plan has no equivalent of. This is the columnar, vectorized execution path rather than an indexed row-store lookup:

```
Join (JOIN FillRightFirst)
| oi[92149] JOIN o[cache-1294] JOIN (p[200] JOIN c[8])
| Type: inner | Strictness: all | Algorithm: SpillingHashJoin(HashJoin)
| Result rows: 3793
```

ClickHouse's `EXPLAIN` is a logical plan only — it does not report execution time the way PostgreSQL's `ANALYZE` does — so wall-clock timing was the comparison axis used for actual performance, with the plans used to explain *why* the numbers land where they do. The net takeaway: for this data volume, PostgreSQL's targeted index lookups are competitive with or faster than ClickHouse's full-scan + `FINAL` deduplication approach on three of four queries; `FINAL` is required for CDC correctness here, and it is not free.

6. Part 5 — Real-Time Dashboard (optional)

A lightweight Streamlit app (`dashboard/app.py`) connects directly to ClickHouse (service name `clickhouse`, credentials from the shared `.env`) and auto-refreshes every 5 seconds, showing: a live count of active orders, revenue over time, and an order-status breakdown — all queried with `FINAL` and the `_peerdb_is_deleted = 0` filter, consistent with the rest of the analytical layer. It was confirmed serving (`HTTP 200` on port 8501) as part of the same end-to-end run used to verify the whole stack for this report.

7. Conclusions

- A fully automated CDC pipeline from PostgreSQL to ClickHouse via PeerDB is achievable with zero manual setup steps, provided the mirror-creation step is scripted and gated on service health rather than done through the UI.
- ClickHouse's `ReplacingMergeTree` + `FINAL` is the correct mechanism for reconciling CDC's append-only UPDATE/DELETE representation with OLAP query needs, but the version column must be one PeerDB guarantees monotonic on every change — not an application-level `updated_at`, which fails silently on deletes. PeerDB's soft-delete convention (`_peerdb_is_deleted`) must be filtered explicitly in every query touching CDC-replicated tables.
- On this dataset size (~200K operations, a few thousand settled rows per query), PostgreSQL's indexed row-store access was competitive with ClickHouse's columnar scan + CDC deduplication overhead for most of our queries; ClickHouse's architectural advantages for OLAP workloads are expected to become more decisive at larger data volumes, where PostgreSQL's row-by-row heap access and lack of vectorized execution scale worse than ClickHouse's columnar, batch-oriented execution.

Appendix: Running the project

```
git clone https://github.com/pyrawn/Real-time-Analytics-CDC-PostgreSQL-Clickhouse.git
cd Real-time-Analytics-CDC-PostgreSQL-Clickhouse
cp .env.example .env
docker compose up --build -d
docker compose logs peerdb-init          # ends with "CDC setup completed!"
docker compose logs --tail 5 generator  # total_ops should keep growing
```

PeerDB UI: <http://localhost:3000>. Dashboard: <http://localhost:8501>. Full details, including Windows-specific notes, in the project [README.md](#).